

MANUAL DE INSTALACION Y CONFIGURACION

VETERINARIA WEB

Sistema de gestion de citas veterinarias

Steven Velásquez Toro

Juan Diego Cano Moncada

Pruebas y Gestión de la Seguridad

Septiembre de 2026

1. Introduccion

Veterinaria Web es una aplicacion para la gestion de citas de una clinica veterinaria. Este manual explica como instalarla y ponerla en funcionamiento, tanto en un computador personal para desarrollo como en un servidor accesible desde internet.

No hace falta experiencia previa con Java ni con Spring Boot. Cada paso trae el comando que hay que escribir y la forma de comprobar que salio bien. Los capitulos 2 a 6 cubren la instalacion local; el 7 explica el despliegue en el servidor y el 8 reúne los errores mas comunes con su solucion.

1.1 Como esta construida la aplicacion

El proyecto genera un unico archivo ejecutable que incluye el servidor web, de modo que no hay que instalar Tomcat ni ningun otro servidor por separado. Las pantallas se arman en el servidor con Thymeleaf, asi que tampoco existe un proyecto de frontend aparte. Los datos se guardan en PostgreSQL.

Tabla 1. Tecnologias utilizadas

Componente	Version
Java	21
Spring Boot	4.1.1
Spring Security	7.1.1
Hibernate / Spring Data JPA	7.4.5
Thymeleaf	3.1.5
PostgreSQL	17 (local) / 14 (servidor)
Maven	Incluido en el proyecto
JUnit	5

2. Requisitos previos

Antes de empezar hay que tener instalado lo siguiente:

Tabla 2. Software necesario

Programa	Version	Para que sirve
JDK de Java	21	Compilar y ejecutar la aplicacion
PostgreSQL	17	Guardar los datos
Git	Cualquiera	Descargar el codigo
Docker	Opcional	Alternativa para levantar la base de datos

Maven no hay que instalarlo. El proyecto trae un archivo llamado mvnw que lo descarga solo la primera vez que se usa.

Para verificar que Java y Git estan bien instalados, abrir una terminal y escribir:

```
java -version
git --version
```

El primer comando tiene que responder con una version 21. Si responde con una anterior o dice que no reconoce el comando, hay que instalar el JDK 21 antes de seguir.

3. Instalar PostgreSQL

Estos pasos son para Windows. En Linux o Mac se puede usar el gestor de paquetes del sistema.

3.1 Descargar e instalar

Ir a [postgresql.org/download/windows](https://www.postgresql.org/download/windows) y descargar el instalador de la version 17 para Windows x86-64. Durante la instalacion:

Dejar marcados PostgreSQL Server, pgAdmin 4 y Command Line Tools. Stack Builder se puede desmarcar.

Anotar la contraseña que se le pone al usuario postgres, porque hace falta en el paso siguiente.

Dejar el puerto 5432 y la configuracion regional que vienen por defecto.

3.2 Crear la base de datos

Abrir SQL Shell (psql) desde el menu de inicio. Va a preguntar servidor, base, puerto y usuario: a todo eso se le da Enter para aceptar lo que propone. Cuando pida la contraseña, escribir la del usuario postgres.

Ya adentro, ejecutar estas cinco lineas:

```
CREATE DATABASE veterinaria;
CREATE USER veterinaria WITH PASSWORD 'veterinaria';
GRANT ALL PRIVILEGES ON DATABASE veterinaria TO veterinaria;
\c veterinaria
GRANT ALL ON SCHEMA public TO veterinaria;
```

Nota: La ultima linea no se puede omitir. Desde PostgreSQL 15, un usuario que no tenga ese permiso no puede crear tablas, y la aplicacion falla al arrancar con el mensaje "permission denied for schema public". Es uno de los errores mas frecuentes.

3.3 Comprobar que quedo bien

Salir con \q y volver a entrar, esta vez escribiendo veterinaria como base de datos y como usuario. Ya adentro, probar:

```
CREATE TABLE prueba (id int);
DROP TABLE prueba;
```

Si las dos lineas responden sin error, el usuario tiene los permisos que necesita.

3.4 Alternativa con Docker

Si se prefiere no instalar PostgreSQL, el proyecto incluye un archivo docker-compose.yml que levanta la base con los mismos datos de acceso:

```
docker compose up -d
docker compose ps
```

4. Descargar el proyecto

```
git clone https://github.com/DbodoR/Veterinaria-web.git
cd Veterinaria-web
```

La rama main tiene la version estable, que es la que esta publicada. La rama develop recibe las funcionalidades nuevas antes de integrarlas.

4.1 Como esta organizado el codigo

Tabla 3. Estructura de carpetas

Carpeta	Que contiene
src/main/java/.../model	Entidades que representan las tablas
src/main/java/.../repository	Consultas a la base de datos
src/main/java/.../security	Login y control de acceso
src/main/java/.../controller	Controladores de cada pantalla
src/main/java/.../config	Configuracion y datos iniciales
src/main/resources/templates	Paginas HTML
src/main/resources/static	Estilos e imagenes
src/test/java	Pruebas automaticas
Dockerfile	Instrucciones para construir la imagen del servidor

5. Configuracion

La configuracion esta repartida en cuatro archivos: uno con lo que comparten todos los entornos y tres especificos. Cual se usa depende de la variable `SPRING_PROFILES_ACTIVE`; si no se indica nada, se usa dev.

Tabla 4. Archivos de configuracion

Archivo	Perfil	Cuando se usa
application.properties	—	Siempre
application-dev.properties	dev	En el computador de desarrollo
application-prod.properties	prod	En el servidor publico
application-test.properties	test	Al ejecutar las pruebas

5.1 Configuracion local

Si se respetaron los nombres del capitulo 3, no hay que cambiar nada. El archivo `application-dev.properties` ya viene asi:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/veterinaria
spring.datasource.username=veterinaria
spring.datasource.password=veterinaria
spring.jpa.hibernate.ddl-auto=update
```

La ultima linea hace que Hibernate cree las tablas solo, a partir de las clases del proyecto. No borra datos existentes.

5.2 Datos de prueba

En modo desarrollo, el archivo data.sql carga al arrancar un administrador, un cliente y cinco servicios de ejemplo. Se puede ejecutar todas las veces que haga falta sin duplicar registros.

En el servidor este archivo no se ejecuta. Allí la cuenta de administrador la crea el componente AdminInicialSeeder, que se explica en el capítulo 7.

6. Compilar y ejecutar

6.1 Compilar

```
mvnw clean package
```

La primera vez tarda varios minutos porque descarga las librerías. Al terminar tiene que decir BUILD SUCCESS y aparecer el archivo target\Veterinaria-web-0.0.1-SNAPSHOT.jar.

En Linux y Mac hay que escribir ./mvnw en vez de mvnw.

6.2 Ejecutar

Para trabajar en el proyecto:

```
mvnw spring-boot:run
```

O ejecutando directamente el archivo ya compilado:

```
java -jar target\Veterinaria-web-0.0.1-SNAPSHOT.jar
```

La aplicacion queda en http://localhost:8080. Esta lista cuando en la consola aparece el mensaje "Started VeterinariaWebApplication".

Nota: Si el puerto 8080 esta ocupado por otro programa, se puede usar otro agregando --server.port=8081 al final del comando.

6.3 Usuarios de prueba

Tabla 5. Credenciales del entorno de desarrollo

Tipo	Correo	Contraseña	Rol
Administrador	admin@veterinaria.com	Admin123	ADMINISTRADOR
Cliente	cliente@correo.com	Cliente123	CLIENTE

Estos usuarios solo existen en el entorno local. En el servidor publico no estan.

6.4 Ejecutar las pruebas

El proyecto tiene pruebas automaticas que verifican los criterios de aceptacion. Usan una base en memoria, asi que no necesitan que PostgreSQL este encendido ni modifican los datos locales.

```
mvnw clean test
```

El resultado esperado es:

```
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0  
[INFO] BUILD SUCCESS
```

Maven deja un informe detallado en la carpeta target\surefire-reports.

7. Publicar en un servidor

La aplicacion esta publicada en Render, que construye la imagen a partir del Dockerfile del repositorio. Aqui se explica como repetir el proceso desde cero.

7.1 Crear la base de datos

En el panel de Render, presionar New y elegir Postgres. Llenar el formulario asi:

Tabla 6. Datos para crear la base en Render

Campo	Valor
Name	veterinaria-db
Database	veterinaria
User	veterinaria
Region	La misma que se le va a dar al servicio web
Instance Type	Free

Nota: Render le agrega un sufijo al nombre de la base. Aunque se escriba "veterinaria", el nombre real puede terminar siendo "veterinaria_rtz". Una vez creada hay que entrar a la seccion Connections y usar el nombre que aparece ahi, no el que se pidio.

7.2 Crear el servicio web

Presionar New, elegir Web Service y conectar el repositorio de GitHub. Configurar la rama main, el lenguaje Docker y el tipo de instancia Free.

7.3 Variables de entorno

En la seccion Environment del servicio hay que agregar estas seis variables:

Tabla 7. Variables de entorno del servidor

Variable	Valor
SPRING_PROFILES_ACTIVE	prod
DB_URL	jdbc:postgresql://HOSTNAME/NOMBRE_DE_LA_BASE
DB_USER	El que aparece en Connections
DB_PASSWORD	La que aparece en Connections
ADMIN_CORREO	admin@veterinaria.com
ADMIN_PASSWORD	Una contraseña segura elegida por el equipo

Nota: Render muestra la direccion de conexion en el formato postgresql://usuario:clave@host/base, que Spring no entiende. La variable DB_URL hay que armarla a mano: empezando con jdbc: y sin incluir usuario ni contraseña, que van en variables aparte.

La variable PORT no hay que crearla. Render la genera sola y la aplicacion ya esta preparada para leerla.

7.4 Como se crea el administrador

El componente AdminInicialSeeder solo funciona con el perfil prod. Al arrancar revisa si ya hay algun administrador activo; si no lo hay, crea uno con el correo y la contrasena de las variables de entorno, guardando la contrasena cifrada.

De esta forma ninguna contrasena del servidor queda escrita en el repositorio. Si ADMIN_PASSWORD no esta definida, el componente no crea nada y deja un mensaje de error en el log.

7.5 Verificar el despliegue

En la pestana Logs tienen que aparecer, en este orden:

```
The following 1 profile is active: "prod"
HikariPool-1 - Start completed
Administrador inicial creado con el correo admin@veterinaria.com
Started VeterinariaWebApplication
==> Your service is live
```

Cuando aparece la ultima, el estado cambia a Live y se muestra la direccion publica.

Nota: En el plan gratuito el servicio se apaga si pasa quince minutos sin visitas. La primera vez que alguien entre despues de eso, la pagina puede tardar cerca de un minuto en cargar. No es un error de la aplicacion.

7.6 Actualizaciones

Render revisa la rama main. Cada vez que algo se integra ahi, construye y publica la nueva version automaticamente.

8. Errores frecuentes

Estos son los problemas que encontro el equipo durante la instalacion y el despliegue.

Tabla 8. Errores y su solucion

Mensaje	Por que pasa	Como se soluciona
Port 8080 was already in use	Otro programa esta usando el puerto	Agregar --server.port=8081 al comando
permission denied for schema public	Al usuario le falta permiso sobre el esquema	Ejecutar GRANT ALL ON SCHEMA public TO veterinaria
FATAL: database "..." does not exist	El nombre de la base no es el que se puso en DB_URL	Buscar el nombre real en Connections y corregirlo
failed to read dockerfile	El Dockerfile no esta en la rama que Render construye	Verificar que el archivo llego a main
Connection refused	PostgreSQL no esta corriendo	Iniciar el servicio postgresql-x64-17 en services.msc
constraint ... does not exist, skipping	Aviso normal de Hibernate la primera vez	No hay que hacer nada
No se puede entrar con la contraseña de admin	ADMIN_PASSWORD no estaba puesta cuando se creo la cuenta	Definir la variable, borrar el usuario y reiniciar

9. Resumen de comandos

Tabla 9. Comandos de uso frecuente

Para	Comando
Descargar el proyecto	<code>git clone https://github.com/DbodoR/Veterinaria-web.git</code>
Compilar	<code>mvnw clean package</code>
Ejecutar	<code>mvnw spring-boot:run</code>
Ejecutar el jar	<code>java -jar target\Veterinaria-web-0.0.1-SNAPSHOT.jar</code>
Usar otro puerto	<code>java -jar ... --server.port=8081</code>
Correr las pruebas	<code>mvnw clean test</code>
Levantar la base con Docker	<code>docker compose up -d</code>
Apagar la base	<code>docker compose down</code>

Tabla 10. Direcciones de la aplicacion

Entorno	Direccion
Local	<code>http://localhost:8080</code>
Publicado	<code>https://veterinaria-web-i88k.onrender.com</code>